



Computer Science Department
Web Application and Technologies (COMP 334)
Summer Semester 2025/2026

Assignment 3: PHP

Due Date 17/09/2026

1. Overview

This assignment is a continuation of Assignment 2, in which you built a fully dynamic, database-driven Maintenance Request Ticketing System using PHP and MySQL.

In this assignment, you will enhance the visual appearance and layout of the web application by writing and applying CSS rules to the pages produced by your Assignment 2 application.

You must use your existing Assignment 2 project as the starting point. The functionality of the application, including authentication, searching, viewing tickets, assigning tickets, submitting maintenance requests, and database operations, must continue to work correctly.

The main purpose of this assignment is to practice CSS concepts and apply them to a complete web application.

Important Constraints

- Single external CSS file: Create one external stylesheet named style.css. Do not use embedded `<style>` tags or inline style attributes.
- Course CSS only: CSS frameworks such as Bootstrap, Tailwind, or any other external CSS library are not allowed.
- No JavaScript: You are not required to use JavaScript for this assignment.
- Clean CSS file: Remove any CSS rule or class that is not actually used in your HTML/PHP pages.
- You may modify your PHP/HTML markup when necessary to add appropriate CSS classes or IDs, but you must not change the required functionality of Assignment 2.

2. Linking Your Stylesheet

Create an external stylesheet named:

style.css

The stylesheet must be linked to all pages of your application.

For example:

```
<link rel="stylesheet" href="style.css">
```

Place style.css in the main folder of your Assignment 3 application.

All pages generated by the application should use the same stylesheet to maintain a consistent visual design.

Task 1 — Overall Page Layout

Create a consistent layout for all pages of the Maintenance Request Ticketing System.

The main page structure should contain:

- Header
- Navigation
- Main content
- Footer

Use CSS to control:

- Page width
- Maximum width
- Margins
- Padding
- Background
- Font
- Spacing

The main content should be centered on larger screens and should use the available screen width appropriately.

Use the CSS box model to create appropriate spacing between the different sections of the page.

Task 2 — Header Styling

Style the `<header>` section of the application.

The header should contain the application title/logo and the navigation area.

Requirements:

- Apply a suitable background colour to the header.
- Use Flexbox to arrange the logo/title and other header elements.
- Display the application title clearly.
- Use appropriate font size and font weight for the title.
- Add suitable padding around the header.
- Style the navigation links using CSS.
- Display the navigation links horizontally using Flexbox.
- Add consistent spacing between navigation links.
- Apply a `:hover` effect to navigation links.
- Style the logged-in user's name and role differently from normal navigation links.

The final design should make it clear whether the logged-in user is a Manager or Customer.

Task 3 — Footer Styling

Style the `<footer>` section of all pages.

The footer should contain the information required in Assignment 2, including company contact information, Contact Us link, copyright information, your name, your ID, and the link to your home page.

Requirements:

- Apply a suitable background colour.
- Add appropriate padding.
- Arrange the footer content using Flexbox.
- Clearly separate contact information from the copyright information.
- Style the <address> element appropriately.
- Style the footer links.
- Apply a hover effect to footer links.
- Ensure that the footer has a consistent appearance throughout the application.

Task 4 — Login Page Styling

Style the login page.

The login form is submitted to login.php in Assignment 2.

Requirements:

- Center the login form within the page.
- Place the form inside a visually distinguishable container/card.
- Apply a maximum width to the form.
- Style the labels.
- Style the email/username and password fields.
- Add appropriate padding and spacing.
- Apply borders and border-radius.
- Use the :focus pseudo-class for input fields.
- Style the Login button.
- Add a hover effect to the Login button.
- Style error messages clearly.

Task 5 — Dashboard Layout

Style the Manager Dashboard and Customer Dashboard.

The Manager Dashboard contains a filter/search form and a Tickets List table.

The Customer Dashboard provides customers with access to their own tickets and the maintenance request functionality.

Requirements:

Use CSS Grid or Flexbox to create a clear dashboard layout.

The dashboard should contain visually separated areas for:

- Page heading
- Search/filter section
- Ticket list
- Other available actions

The layout should have appropriate:

- Width
- Padding
- Margin
- Gap
- Borders
- Background
- Border radius

The dashboard must remain usable when the browser window becomes smaller.

Task 6 — Search / Filter Form Styling

Style the filter/search form used by the Manager Dashboard.

The form contains search/filter fields for ticket information such as description, customer name, submission date, status, and emergency level.

Requirements:

- Arrange the form controls using Flexbox or Grid.
- Apply consistent spacing between form elements.
- Style text inputs.
- Style `<select>` elements.
- Style radio buttons appropriately.
- Style the Filter button.
- Style the Reset/Show All functionality if available.
- Apply a hover effect to buttons.
- Apply a `:focus` effect to input fields.
- Give the filter area a visually distinguishable background.

The form should be easy to understand and use.

Task 7 — Tickets Table Styling

Style the Tickets List table displayed on the Manager Dashboard.

Requirements:

The table should have:

- A clearly styled header row.
- Appropriate padding inside table cells.
- Borders or other visual separation between rows.
- Alternating row backgrounds using `:nth-child()` or another appropriate CSS technique.
- A hover effect when the mouse moves over a row.
- Appropriate alignment of ticket information.
- Clearly visible action links/images.

The ticket ID should remain visually identifiable as a link to the ticket view page.

Task 8 — Ticket Status and Emergency Level

Use CSS classes to visually distinguish different ticket statuses and emergency levels.

The ticket information in Assignment 2 includes:

- Status

- Emergency Level
- Assigned Staff
- Submission Date
- Assignment Date.

Create CSS classes for the ticket status, such as:

.status-pending
.status-assigned
.status-completed

Create CSS classes for emergency levels, such as:

.emergency-low
.emergency-medium
.emergency-high

Requirements:

- Each status must have a visually distinguishable style.
- Each emergency level must have a visually distinguishable style.
- Use appropriate background/text colours.
- Use padding and border-radius where appropriate.
- The text must remain readable and have sufficient contrast.

The exact colour scheme is your choice.

Task 9 — Ticket View Page Styling

Style the ticket details page generated by view.php.

Assignment 2 requires the ticket information to be displayed dynamically and an error page to be generated when an invalid ticket ID is supplied.

Requirements:

Display the ticket information inside a visually distinguishable container/card.

Style the following information:

- Ticket ID
- Customer Name
- Description
- Submitted Date
- Status
- Emergency Level
- Assigned Staff
- Assigned Date
- Ticket Image

The ticket information should be clearly organized using appropriate CSS layout techniques.

Ticket Image

The ticket image should:

- Have a suitable maximum width and/or height.
- Maintain its aspect ratio.
- Be visually separated from the text.
- Not exceed the available page width.

Task 10 — Assign Ticket Page Styling

Style the page generated by assign.php.

Assignment 2 requires the staff names to be generated from the database and the assignment form to submit the ticket ID and staff ID.

Requirements:

- Center the assignment form.
- Place the form inside a card/container.
- Style the ticket information.
- Style the staff <select> element.
- Style the Assign Ticket button.
- Add a hover effect to the button.
- Style error/information messages appropriately.
- Use consistent styling with the other forms in the application.

Task 11 — Maintenance Request Form Styling

Style the Maintenance Request form generated by request.php.

The form contains fields including Name, Email, Location, Issue Description, Urgency Level, and Upload Photo.

Requirements:

- Center the form within the main content area.
- Give the form a suitable maximum width.
- Use a border, padding, and border-radius.
- Arrange labels and form controls clearly using Flexbox or Grid.
- Style text inputs.
- Style the textarea.
- Style the urgency <select>.
- Style the file upload input.
- Style the Submit Request button.
- Add a hover effect to the submit button.
- Apply a :focus effect to form controls.

The form should be easy to read and use.

Task 12 — Confirmation Page Styling

Style the confirmation page generated after successfully submitting a maintenance request.

Assignment 2 redirects the user to confirm.php after successfully submitting the ticket.

Requirements:

- Display the confirmation message inside a visually distinguishable container.
- Use appropriate spacing.

- Use a clear heading.
- Style the ticket ID or other important information.
- Style the link/button that allows the user to return to the dashboard.
- Apply consistent colours and typography with the rest of the application.

Task 13 — Buttons and Links

Create a consistent style for all buttons and important action links.

The application contains several actions, including:

- Login
- Filter
- View Ticket
- Assign Ticket
- Submit Maintenance Request
- Logout
- Return/Back actions

Requirements:

Create a reusable base button class, for example:

`.btn`

You may also create modifier classes such as:

`.btn-primary`

`.btn-success`

`.btn-danger`

`.btn-secondary`

The exact class names are your choice.

Each button should have:

- Padding
- Border
- Border radius
- Appropriate font
- Cursor styling
- Hover effect

Task 14 — CSS Pseudo-classes

Your stylesheet must demonstrate the use of CSS pseudo-classes.

You must use at least the following:

`:hover`

`:focus`

`:nth-child()`

Examples include:

- Navigation link hover.
- Button hover.
- Input focus.

- Alternating table rows.

You may use additional pseudo-classes where appropriate.

Task 15 — Responsive Design

Make the Maintenance Request Ticketing System responsive.

The application should remain usable on:

- Desktop computers
- Tablets
- Mobile phones

You must use at least one CSS media query.

For smaller screens:

- The page layout should adapt to the available width.
- Navigation links should remain usable.
- Dashboard sections should stack vertically when necessary.
- Tables should remain readable.
- Forms should fit the screen.
- Images should scale appropriately.
- Buttons should remain usable.
- Text should remain readable.

Placement and Submission

File Placement on CS Host

Place the Assignment 3 files in:

`public_html/assignments/assThree/`

Your Assignment 2 PHP application should be updated so that all relevant pages use the new `style.css` stylesheet.

Update your home page:

`index.htm`

by adding a link to the Assignment 3 main page.

ZIP Archive

Compress all Assignment 3 files into a single archive named:

`assThree-stID.zip`

The ZIP file should contain:

- Updated PHP files
- `style.css`
- Images required by the application
- Any other required project files

Submit the ZIP file to ITC before the announced deadline.

The application must also be uploaded and working correctly on the CS Host.